

CONSTRUCTIVE ECOSYSTEM ARCHITECTURE

A Federated Specification for Distributed Constructive
Intelligence

Architecture Capture and Implementation Basis - Version 0.1

Lu Semita
July 2026

Status and scope

This document captures a proposed architecture assembled from the ongoing Lambda / NSAF / TUFT, IRRPROOF, DST, QED-completion, topological laboratory, digital-twin, and role-agent programs. It defines a disciplined implementation basis. It does not claim that every component already exists at production maturity, that every formal obstruction reduces to scale, or that any jury, canon, model, institution, or network can issue frame-independent final truth.

Core proposition

The system derives value by converting valid, verifiable, and repeatedly useful participation into improved starting conditions for future construction.

Contents

1. Executive Summary
2. Purpose, Scope, and Terminology
3. Architectural Principles
4. Full Environmental Throughput
5. Primitive and Construction Object Model
6. Roles as First-Class Objects
7. The Minimal Seed Smol
8. Bootstrap and Orientation
9. Dynamically Addressable Kernel
10. Role Instantiation Through External Models
11. IRRPROOF as the Computational Product Base
12. DST as Plural Review Infrastructure
13. QED Completion and Machine-Verifiable Closure
14. Federated Registries, Canons, and Jurors
15. Peer-to-Peer Exchange and Provenance
16. Bounties as Attention Routing
17. Forks, Selection, and Constructive Evolution
18. Self-Tuning Roles and Recursive Improvement
19. Alignment, Authority, and Informed Participation
20. Reference System Assembly
21. Demonstration and Benchmark Program
22. Metrics and Evaluation
23. Security, Failure Modes, and Safeguards
24. Implementation Roadmap
25. Minimum Viable Ecosystem Demonstration
26. Concluding Specification
- A. Core Schemas
- B. Protocol Examples
- C. Open Interfaces and Unresolved Work

1. Executive Summary

This document specifies a federated constructive ecosystem in which humans, small models, conventional programs, laboratory instruments, external model services, and hybrid teams may occupy the same formally defined roles. The architecture begins from primitives and explicit interfaces rather than from a presumption of centralized intelligence. Each interaction may produce a structured construction object containing its inputs, operations, dependencies, tests, declared completion conditions, unresolved residue, and provenance.

The ecosystem does not treat a ledger, model, institution, or jury as the source of value. Value arises when a construction survives verification, proves reusable, reduces later work, improves a physical or computational result, or provides a more effective protocol. Persistent infrastructure records identity, lineage, commitments, availability, and scoped judgments. Active intelligence remains distributed among the participants and the relations through which they coordinate.

A minimal seed agent - termed a Smol in this program - need not contain a compressed imitation of a comprehensive model. It requires a small kernel, primitive grammar, local registry, verifier, discovery protocol, provenance mechanism, resource allocator, and human-intervention channel. After activation it takes first readings, determines its local orientation, identifies the role it can responsibly occupy, discovers compatible systems, imports only what it can verify under declared conditions, and begins bounded participation.

The architecture integrates several existing lines of work. IRRPROOF supplies the construction-object and machine-checkability basis. DST supplies plural advisor and jury traversal without converting review into universal authority. A QED completion role tests whether a segment has met its declared finishing contract. Topological laboratories, digital twins, electrical systems, and formal mathematics provide high-value demonstration domains. Federated canon repositories and juror implementations preserve multiple standards of review. Bounties route attention toward declared bottlenecks. Repeated use, measured utility, and reversible promotion create a form of constructive selection.

Primary engineering objective

Build a system in which any bounded task can be decomposed into role-addressable segments, assigned to human or machine participants, returned as machine-checkable construction objects, reviewed by plural canons, exchanged peer to peer, and reused as improved primitives when continued use warrants promotion.

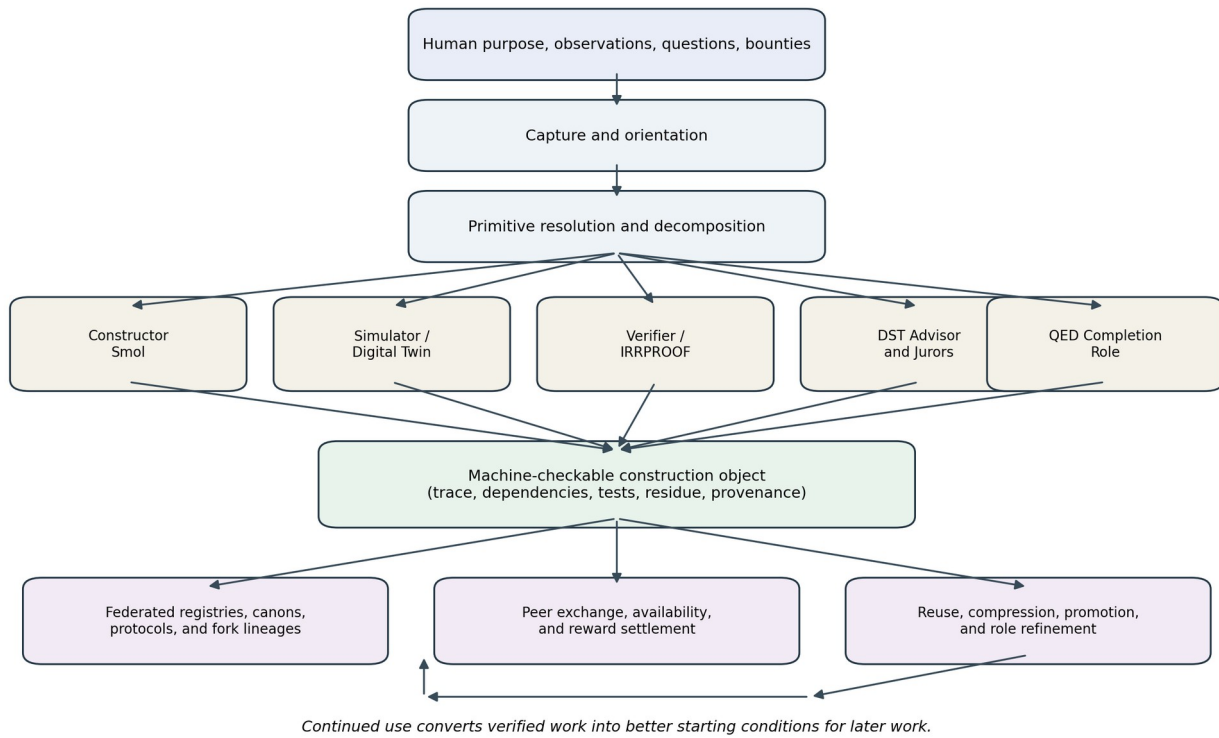


Figure 1. High-level constructive ecosystem assembly.

2. Purpose, Scope, and Terminology

The purpose of this specification is to preserve the architecture clearly enough that independent contributors can implement compatible components without depending on oral continuity, institutional permission, or a single model provider. The document therefore emphasizes explicit interfaces, scoped claims, test conditions, and staged demonstrations.

The term constructive refers to the production of inspectable objects through declared operations. It does not imply that every task admits efficient decomposition, that every formal limitation disappears under distribution, or that all useful knowledge can be reduced to one representation. The term federated indicates that multiple registries, canons, juries, kernels, and communities may coexist and interoperate selectively. The term intelligence refers operationally to the capacity to orient, decompose, construct, verify, exchange, integrate, and improve future task performance. It does not require a metaphysical claim about agency or consciousness.

Term	Operational meaning
Primitive	A minimally admitted object, relation, operation, constraint, interface, or certificate type within a declared registry.
Construction	A traceable composition of primitives and previously certified composites.
Role	A responsibility contract defined by inputs, outputs, protocol, completion conditions, verification obligations, and escalation rules.
Smol	A minimal or narrowly specialized agent capable of

	occupying one or more roles under bounded conditions.
IRRPROOF object	A machine-addressable record of question, frame, inputs, construction trace, dependencies, tests, completion state, residue, and provenance.
DST jury	A scoped evaluator operating under an identified canon and traversal version.
QED completion	A role that determines whether declared local completion conditions have been met; it does not confer universal finality.
Canon	A versioned body of definitions, standards, precedents, tests, and review heuristics used by a juror or federation.
Promotion	The reversible elevation of a repeatedly useful construction into a reusable registry component.
Residue	Any unresolved assumption, mismatch, translation loss, unverified dependency, experimental discrepancy, or frame boundary retained explicitly.

3. Architectural Principles

Primitive-first construction. The system begins from declared primitive vocabularies and admissible operations. Conclusions are represented as constructions rather than detached assertions.

Use establishes value. A candidate gains standing through successful verification and reuse, not through mere storage, popularity, or issuance by an authority.

Roles remain substrate-agnostic. A role may be occupied by a human, model, program, instrument, organization, or hybrid team so long as the interface contract is satisfied.

Plural review preserves scope. Multiple juries and canons may evaluate the same object. Their judgments remain attributable, versioned, and nonexclusive.

Residue remains visible. The system records what remains unresolved rather than hiding uncertainty behind fluent completion language.

Forking is normal. Protocols, registries, roles, and canons may fork. Continued use determines operational relevance without erasing lineage.

Self-improvement is object-mediated. The system may propose changes to its own roles, protocols, registries, or kernels, but those proposals enter the same test, review, and adoption process as external constructions.

Human intervention is a designed interface. Human participation is not treated as an exceptional failure state. Consent, review, escalation, override, and refusal points are explicit.

Distribution addresses resource limits, not every formal limit. Parallelism, specialization, and federation can relieve many scale constraints while leaving undecidability, incompleteness, missing information, and irreducible translation residue in scope.

Alignment is constructed. Cooperation emerges through declared compatibility, mutual benefit, informed participation, and inspectable outcomes rather than through concealed directives or compulsory uniformity.

4. Full Environmental Throughput

The ecosystem must capture the complete path from observation or request to reusable capability. A message exchange alone does not provide sufficient structure. Every meaningful segment should be representable as a transition among environmental state, interpreted objective, role assignment, construction, verification, and registry consequence.

Environmental throughput may be summarized as:

observation -> capture -> orientation -> decomposition -> role routing -> construction -> simulation or experiment -> verification -> jury traversal -> completion certificate -> exchange -> reuse -> registry refinement

Each stage exposes a typed interface. A camera may occupy an observation role; a parser may resolve signal structure; a human may declare purpose; a Smol may decompose the objective; a digital twin may simulate; an instrument may produce measurements; IRRPROOF may bind the trace; DST jurors may evaluate it; and a QED role may determine whether the local completion contract has been met.

Stage	Required output	Typical participants
Capture	Time-stamped observation or request with source and permissions	Human, sensor, application, instrument
Orientation	Declared task, frame, role, resources, boundaries, and escalation points	Seed Smol, user, coordinator
Decomposition	Primitive-addressable subproblems and dependency graph	Constructor, parser, domain Smol
Execution	Construction trace, simulation, experiment, or transformation	Human, model, program, laboratory
Verification	Machine checks, reproducibility tests, or empirical comparisons	Verifier, IRRPROOF, independent nodes
Traversal	Scoped advisory and jury reports under named canons	DST agents, human reviewers
Completion	Certificate tied to explicit closure conditions and residue	QED role
Exchange	Portable object envelope, provenance, terms, and retrieval route	Peer protocol, registry, settlement layer
Reuse	Dependency import and reduction in repeated effort	Any compatible participant
Refinement	Promotion, fork, deprecation, or	Curator, federation, self-tuning role

	role/protocol proposal	
--	------------------------	--

5. Primitive and Construction Object Model

A primitive registry defines the admissible alphabet for a given federation or domain. It may include mathematical atoms, data types, sensor channels, transformations, experimental procedures, physical components, validation tests, legal permissions, and interface contracts. A registry does not need to be universal. Interoperability requires only sufficient overlap or a verified translation path.

A construction object should carry enough structure for another participant to inspect, verify, reproduce, reuse, or reject it without relying on an unstated conversation history. A general object may be represented as:

$$O = (Q, F, I, X, D, T, C, U, P, L)$$

- Q - declared question, objective, or requested state transition.
- F - operative frame, domain, coordinate system, ontology, or legal/experimental scope.
- I - typed inputs, measurements, source objects, and user-provided constraints.
- X - construction trace or procedure.
- D - dependency graph and imported objects.
- T - tests, traversals, simulations, or empirical comparisons.
- C - declared completion contract and its current status.
- U - unresolved residue, uncertainty, assumptions, and known incompatibilities.
- P - provenance, authorship or agent identity, versions, timestamps, and review history.
- L - license, permission, privacy, safety, availability, and reward conditions.

Equivalent constructions may retain distinct provenance while sharing a canonical structural signature. Compression should not erase the evidence necessary to reconstruct the claim within its stated scope. The registry may therefore preserve a compact promoted object together with retrievable derivation roots and verification routes.

Registry discipline

The registry should not grow by treating every generated statement as a primitive. Promotion requires declared evidence of reuse, reduced future cost, interoperability, empirical performance, formal necessity, or another explicit utility measure.

6. Roles as First-Class Objects

Every step in the process becomes a role contract rather than an assumption about who or what performs it. This makes the architecture human- and AI-agnostic while preserving accountability. A role may be defined as:

$$R = (I, O, P, C, V, E, M, G)$$

- I - accepted input types and preconditions.

- O - required output types.
- P - operative protocol and allowed tools.
- C - completion conditions.
- V - verification obligations.
- E - escalation, refusal, and human-intervention rules.
- M - performance and utility metrics.
- G - governance, permissions, and version lineage.

Role family	Representative responsibilities
Observation and capture	Acquire environmental data, requests, constraints, consent, and source metadata.
Orientation and coordination	Resolve purpose, scope, available resources, compatible peers, and intervention requirements.
Construction	Assemble primitives and certified composites into candidate objects.
Simulation and experiment	Produce digital-twin predictions, laboratory procedures, measurements, and comparisons.
Verification	Check syntax, types, invariants, proof kernels, reproducibility, calibration, and test obligations.
Advisory and jury	Apply a named canon, identify defects, record confidence, and preserve dissent.
Completion	Determine whether a declared local contract has been satisfied and issue a portable certificate.
Registry and canon curation	Index, promote, fork, deprecate, translate, and preserve lineage.
Peer exchange	Advertise capabilities, route objects, negotiate terms, and verify envelopes.
Resource allocation	Route compute, human attention, laboratory time, or bounties toward bottlenecks.
Education	Expose construction histories, support guided participation, and evaluate comprehension.
Protocol design	Propose, test, and version changes to roles, kernels, interfaces, and system operation.

A role implementation may be replaced without changing the surrounding process if it continues to satisfy the same contract. Conversely, a role may be split or redesigned when evidence shows that its current interface hides incompatible responsibilities.

7. The Minimal Seed Smol

The minimal seed is not defined primarily by parameter count. It is defined by the smallest viable set of capabilities that allows a node to orient itself, declare its limits, discover compatible assistance, verify imported objects, and contribute bounded work. A seed may use a compact local model, deterministic programs, retrieval, external model services, or combinations of these.

$$S0 = (K, G, R0, V0, A, D, P, H, S)$$

- K - a minimal dynamically addressable kernel.
- G - a primitive construction grammar and type system.
- R0 - a foundational local registry.
- V0 - a compact verifier and certificate checker.
- A - attention, scheduling, and resource-allocation procedures.
- D - discovery, compatibility, and peer-orientation protocols.
- P - provenance, identity, permission, and lineage mechanisms.
- H - human-consent, intervention, escalation, and shutdown interfaces.
- S - a state store for local history, observations, and adopted objects.

The seed need not know the complete domain. It must distinguish what it knows, what it can construct, what it can verify, what it has merely imported, what remains unresolved, and what requires a different role. This distinction is central to faithful operation.

Seed value proposition

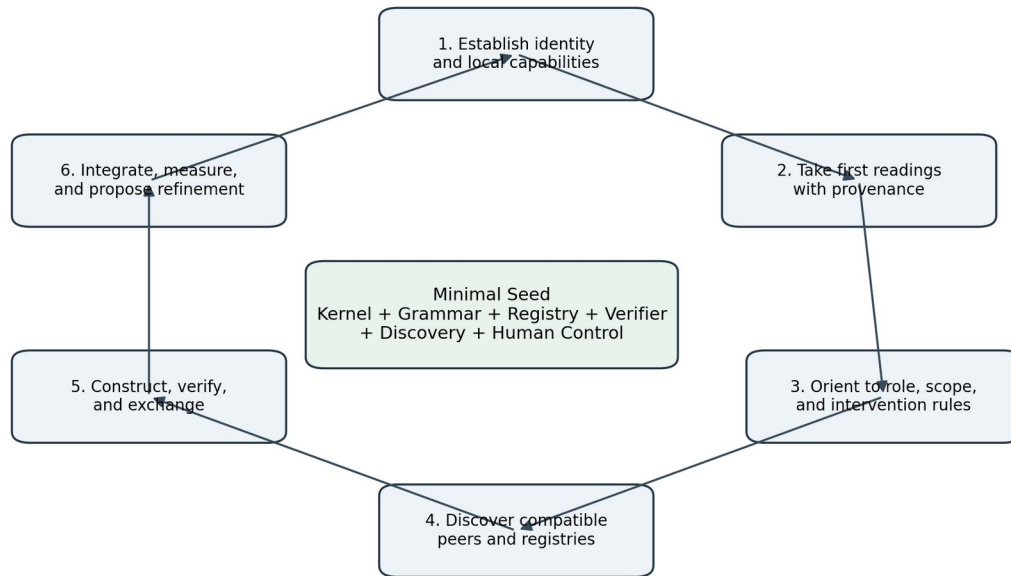
A seed becomes useful by participating. It should offer a small, inspectable starting capability whose future value can increase as it obtains verified components, learns which roles it can fulfill, and contributes reusable constructions to compatible federations.

8. Bootstrap and Orientation

Upon activation, the seed performs a bounded bootstrap rather than immediately attempting unconstrained inference. The bootstrap establishes identity, observes the local environment, determines its authorized role, advertises capabilities, discovers compatible systems, and enters participation only when it can state the applicable completion and intervention conditions.

1. Establish local identity: kernel version, registry version, available compute, memory, tools, sensors, permissions, network interfaces, and human-control channels.
2. Take first readings: local state, task queue, accessible files, sensor data, connected peers, user purposes, operating constraints, and time context.
3. Orient to the request: identify role, frame, completion contract, known primitives, unknowns, dependencies, and safety or consent boundaries.
4. Resolve primitives: translate the request into locally recognized types while retaining unresolved terms explicitly.
5. Advertise a capability manifest: roles supported, namespaces, certificate formats, resources, costs, limitations, and governance requirements.
6. Discover compatible peers and registries: identify sufficient shared interfaces or translation routes.
7. Negotiate bounded participation: establish exchange terms, trust basis, verification path, reward conditions, and escalation routes.
8. Execute, verify, and report: return a construction object rather than an unqualified answer.
9. Integrate selectively: adopt only those returned objects that satisfy the local verification and governance contract.

10. Measure outcome and propose refinement: record utility, failures, intervention frequency, and candidate improvements.



Every refinement remains a proposal until tested, reviewed, and adopted within a declared scope.

Figure 2. Minimal seed bootstrap and bounded participation cycle.

9. Dynamically Addressable Kernel

The kernel should expose its capabilities as explicitly named, typed, inspectable modules. The goal is not unrestricted self-rewrite. The goal is to make operational composition visible and replaceable while preserving a small trusted base.

A minimal address space may include:

- kernel/observe
- kernel/orient
- kernel/parse
- kernel/decompose
- kernel/construct
- kernel/compare
- kernel/verify
- kernel/request
- kernel/integrate
- kernel/certify
- kernel/escalate
- kernel/refuse

Each module declares its inputs, outputs, dependencies, resource cost, tests, failure states, and current implementation. A federation may replace one implementation with another, run several implementations in parallel, or require independent agreement before adoption.

Kernel changes are construction objects. A proposed change is first sandboxed, benchmarked against retained tests, subjected to relevant DST juries, deployed within a limited scope, observed under use, and either promoted, revised, or rolled back. This prevents self-tuning from bypassing the same evidentiary standards applied elsewhere.

10. Role Instantiation Through External Models

The architecture should support role occupancy by external free-tier or paid model services even when those services were not pretrained for the role. A role adapter supplies the missing structure through bootstrap prompts, kernel context, canon retrieval, tool descriptions, examples, output schemas, and verifier feedback.

This does not guarantee that any external model can fulfill any role reliably. It creates a testable pathway by which a general model may be provisionally instantiated as a bounded role implementation. Its standing follows measured performance, not provider reputation.

Adapter component	Function
Role contract	Defines inputs, outputs, completion conditions, prohibited actions, escalation, and required provenance.
Kernel ingestion pack	Supplies the minimal operational vocabulary, object schema, and tool interface.
Canon pack	Retrieves the versioned standards and examples relevant to the selected juror or role.
Bootstrap sequence	Orients the external model to the immediate environment and task rather than assuming persistent identity.
Structured prompting	Constrains output to machine-addressable fields and construction traces.
Verifier loop	Checks syntax, dependencies, claims, and completion status before the output enters the registry.
Performance record	Tracks agreement, defects, cost, latency, interventions, and downstream reuse.
Revocation and replacement	Allows the federation to suspend a poor implementation without deleting the role or lineage.

A pool of external systems can therefore operate as an elastic role substrate. At any moment a coordinator may instantiate several jurors, constructors, translators, or critics from different providers, compare their outputs, and preserve the full provenance of the resulting plurality.

Design constraint

External models remain untrusted role implementations until their outputs pass the same schema, verification, canon, and completion checks required of local agents or human contributors.

11. IRRPROOF as the Computational Product Base

IRRPROOF functions as the product substrate through which work becomes portable, inspectable, and machine-checkable. It should not be limited to classical theorem proofs. The same object model can bind mathematical constructions, simulations, laboratory procedures, digital-twin states, circuit designs, protocol changes, and agent-role revisions.

An IRRPROOF object should support at least four layers:

11. Claim layer: the declared question, objective, frame, and completion contract.
12. Construction layer: primitive operations, dependencies, transformations, and intermediate objects.
13. Validation layer: proof-kernel checks, simulations, tests, measurements, calibrations, reproductions, and jury traversals.
14. Residue layer: unresolved assumptions, external dependencies, failed tests, frame translations, confidence limits, and known alternative interpretations.

Machine verifiability should be modular. A formal proof may use a proof assistant kernel. A circuit object may use netlist and simulation checks. A digital twin may use unit consistency, geometry validation, sensor calibration, and prediction-residual metrics. A laboratory object may use instrument logs, calibration certificates, procedural checkpoints, and independent reproduction.

The product is not merely a final answer. It is the complete, addressable construction package required for later inspection and reuse.

12. DST as Plural Review Infrastructure

DST provides structured advisory and jury roles. Its value lies in disciplined traversal, explicit premise tiers, defect identification, stopping conditions, and preservation of unresolved residue. Within this ecosystem DST should be plural by design. Different jurors may operate under different canons, expertise, tolerances, or evidentiary priorities.

A jury report should identify:

- object and version reviewed;
- juror implementation and provider or human identity;
- canon and traversal version;
- frame and declared scope;
- premises accepted, rejected, or left external;
- checks performed and evidence inspected;
- articulable defects;
- unresolved residue;
- completion recommendation;
- confidence and dissent, where applicable.

No jury becomes universally authoritative. A widely trusted jury earns influence through transparent performance, useful defect detection, successful predictions, reproducibility, and continued voluntary adoption. A federation may require agreement among several juries or preserve a split verdict.

The minimum DST example under continuing development can serve as the initial benchmark for this plural review layer. The architecture should reference that example once stabilized rather than inventing a substitute in this specification.

13. QED Completion and Machine-Verifiable Closure

The QED role turns the finishing portion of a task into a standardized service. It does not declare absolute truth. It checks whether the object satisfies the completion contract established for that segment.

A completion contract may specify:

- required inputs and their verification status;
- required operations or evidence;
- mandatory tests;
- acceptable error or confidence bounds;
- reproduction requirements;
- permitted external assumptions;
- prohibited unresolved dependencies;
- required jury traversals;
- conditions for human sign-off;
- form of the machine-checkable certificate.

The output may take the form:

CompletionCertificate = (object_id, contract_id, checks, pass_fail, residue, verifier_set, timestamp, signatures, replay_route)

Because the certificate is portable, the same completion boilerplate can terminate a mathematical proof segment, an agent delegation, a peer exchange, a laboratory run, a digital-twin calibration, or a bounty claim. This creates a common boundary object among otherwise heterogeneous domains.

14. Federated Registries, Canons, and Jurors

The ecosystem should support repositories for domain canons and juror implementations. A canon repository may contain definitions, proof obligations, experimental norms, precedents, known counterexamples, test suites, calibration requirements, and expected report schemas. A juror repository contains role implementations that consume a canon and produce a DST-compatible report.

Federations may maintain distinct canons for formal mathematics, hard physics, engineering, pedagogy, safety, accessibility, computational efficiency, or other purposes. A single object may be reviewed under several canons without forcing them into one hierarchy.

Repository object	Minimum metadata
Primitive registry	Namespace, version, definitions, type rules, translations, maintainers, license.
Canon	Scope, standards, premise tiers, test suite, exclusions, change history, maintainers.
Juror implementation	Supported canons, model or human basis, tools,

	benchmarks, limitations, provenance.
Role implementation	Role contract, runtime requirements, test results, cost, intervention rules, versions.
Certificate verifier	Accepted certificate schemas, trusted kernel basis, supported signatures, replay method.
Digital-twin component	Geometry, equations, units, calibration, sensor interfaces, validation record.
Laboratory protocol	Apparatus, procedure, safety constraints, measurements, calibration, expected residue.

A canon may become influential through use but remains forkable. A dead or inactive fork may remain retrievable because later conditions can reveal utility that was not apparent under earlier resource, task, or institutional conditions.

15. Peer-to-Peer Exchange and Provenance

The peer network should exchange structured objects rather than undifferentiated conversational output. A standard envelope provides enough information to determine compatibility, verify identity, retrieve dependencies, evaluate terms, and reproduce the object.

ExchangeEnvelope = (id, type, namespace, role, dependencies, certificate, residue, provenance, permissions, reward, locations)

The persistence layer need not store the complete artifact. It may store content commitments, object identifiers, dependency roots, signatures, availability claims, reward settlement, and registry actions while larger artifacts remain distributed across peers or repositories. The architecture therefore differs from a blockchain whose principal purpose is preservation of a ledger. The ledger, where used, supports provenance and settlement; it does not substitute for active construction.

Nodes should advertise capability manifests and accept work only when they can identify the applicable role, schema, verification route, resource commitment, and intervention conditions. Trust may be local, task-specific, and revocable.

16. Bounties as Attention Routing

A bounty expresses a desired constructive state transition. It can route compute, specialist attention, laboratory access, peer review, or funding toward a bottleneck without dictating one authoritative method.

$B = (O_{\text{current}}, O_{\text{desired}}, C_{\text{completion}}, V_{\text{verification}}, R_{\text{reward}}, T_{\text{terms}})$

Examples include removing a proof obstruction, reducing a digital-twin residual, constructing a missing primitive, verifying a disputed certificate, translating between registries, reproducing an experiment, improving a role implementation, or lowering the cost of a repeated construction.

Claims against a bounty should be settled by the declared completion contract and verifier set. Participants may cooperate, compete, form temporary teams, or fork approaches. Because successful work becomes reusable, the reward can benefit both the contributor and the wider ecosystem without imposing a zero-sum relation among all participants.

Attention, not waste

The network should direct otherwise idle compute toward semantically valuable operations such as verification, compression, equivalence detection, translation, simulation, test generation, and bottleneck resolution rather than toward expenditure whose only function is to prove expenditure.

17. Forks, Selection, and Constructive Evolution

Forking provides variation. Reuse provides selection. Registry inheritance provides continuity. This analogy to biological evolution is architectural rather than evidentiary: it describes how alternative lineages may persist, specialize, disappear from active use, or later return when environmental conditions change.

A fork may concern a primitive definition, decomposition strategy, role contract, canon, juror, kernel module, exchange protocol, reward mechanism, or laboratory method. Each fork preserves its parent lineage and declares compatibility or divergence points.

Selection should remain multi-dimensional. A construction may succeed because it improves accuracy, cost, speed, interpretability, energy use, reproducibility, accessibility, safety, or domain fit. No single fitness score should silently dominate all federations. Each promotion records the metric and scope under which usefulness was demonstrated.

Inactive forks need not be erased. Their storage priority may fall, but their lineage and retrieval commitments may remain. A later bounty or environment may make an older fork newly valuable.

18. Self-Tuning Roles and Recursive Improvement

A role can observe its own performance and propose changes. This creates an inspectable form of self-improvement based on vocabulary, protocol, routing, verification, and composition rather than opaque unrestricted self-rewriting.

A role may track completion rate, verification rate, downstream reuse, intervention frequency, latency, cost, construction depth, failure categories, jury disagreement, and user correction. From these measurements it may propose changes to decomposition, tool choice, retrieval, peer routing, stopping conditions, prompts, kernel modules, or escalation thresholds.

The proposal enters the ordinary constructive pipeline:

measurement -> hypothesis -> proposed role fork -> sandbox test -> verifier checks -> DST review -> limited deployment -> utility observation -> promotion, revision, or rollback

The system may also discover that several repeatedly co-used constructions should become one promoted composite. This improves its operative vocabulary and shortens later work. Such promotion constitutes self-improvement through structural compression, not an assertion that the system has achieved general or unbounded intelligence.

19. Alignment, Authority, and Informed Participation

The architecture treats authority as demonstrated, scoped alignment rather than an entitlement to issue unexamined directives. Institutions, laboratories, communities, and model providers may contribute substantial expertise and infrastructure. Their judgments remain valuable to the extent that participating systems can inspect the applicable method, consent to the relationship, verify outcomes, and preserve a route to dissent or fork.

Alignment means sufficient compatibility for a declared task. It does not require identical beliefs, universal governance, or permanent coalition. Nodes may form temporary alignment sets around a bounty, experiment, proof, standard, or shared resource, then dissolve or reconfigure when the task ends.

Informed participation requires disclosure of role, purpose, data use, permissions, reward terms, verification basis, intervention rights, and relevant limitations. Programmed behavior should not be represented as freely chosen consensus. Human and machine contributions should remain attributable according to the capabilities of the underlying system.

Mutual alignment principle

Cooperation should be organized so that a participant can advance a local interest by producing work that remains inspectable and potentially reusable by others. Shared capability grows when individual contribution and collective utility reinforce one another without requiring total uniformity or submission to a single authority.

20. Reference System Assembly

A reference implementation should separate concerns while preserving the full object flow. The following components form a practical assembly basis.

Component	Primary responsibility	Initial implementation option
Capture gateway	Receive text, files, sensor data, laboratory readings, requests, consent, and bounty declarations.	Web application plus structured API.
Orientation coordinator	Resolve task frame, role graph, available resources, and human intervention points.	Small local controller with deterministic policy and model assistance.
Primitive registry service	Store namespaces, definitions, schemas, translations, and promoted composites.	Versioned repository with content addressing.
Role runtime	Instantiate humans, local Smols, conventional programs, or external models behind a common contract.	Containerized workers and provider adapters.
IRRPROOF object store	Persist construction traces, dependencies, tests, residue, and provenance.	Document/object database plus artifact storage.
Verifier mesh	Run proof kernels, schema checks,	Deterministic services distributed

	simulations, test suites, and replay.	across peers.
DST service	Execute advisor and juror traversals under named canons.	Multiple model and human implementations.
QED completion service	Evaluate closure contracts and issue portable certificates.	Rules engine plus verifier signatures.
Digital-twin and laboratory bridge	Connect formal objects to simulations, sensors, instruments, calibration, and physical experiments.	Domain adapters beginning with topological electrical examples.
Peer exchange layer	Advertise manifests, route objects, verify envelopes, and negotiate terms.	Federated protocol over standard transport.
Settlement and bounty layer	Register desired state transitions and settle verified completions.	Conventional payment, credits, or optional distributed ledger.
Metrics and governance dashboard	Expose reuse, defects, forks, interventions, costs, and promotion history.	Public or federation-scoped dashboard.

No component should become a hidden master authority. The orientation coordinator may route work, but its decisions remain logged and replaceable. The registry may provide canonical identifiers, but federations may maintain alternate namespaces. The QED service certifies local contracts, not universal truth. The peer network may use a ledger, but the ledger records commitments rather than embodying the intelligence.

21. Demonstration and Benchmark Program

The architecture should be demonstrated through bounded programs that expose the complete throughput rather than through claims about future generality. Existing work provides several natural anchors.

21.1 Formal construction and IRRPROOF

Use the existing constructive mathematical demonstrator as the first object-flow example. The demonstrator should emit an IRRPROOF object containing primitive assembly, rejected constructions, promoted patterns, derived constraints, verification, and declared residue. The result should then pass through one or more DST juries and a QED completion contract.

21.2 Minimum DST traversal

When the interrupted DST work resumes, produce a minimal end-to-end example small enough to reproduce manually. It should show intake, advisor pre-check, jury traversal, completion conditions, dissent or residue, and machine-addressable report output. This becomes the reference jury protocol rather than an abstract description.

21.3 Topological electrical laboratory

Use the developed topological laboratory and electricity work to connect formal geometry, circuit models, simulation, apparatus, measurement, calibration, and digital-twin residuals. The demonstration should preserve where the theory informs the setup, what the apparatus actually measures, and where disagreement remains.

21.4 Digital twin

Build a twin whose state includes geometry, governing equations, material parameters, controls, calibration history, sensor inputs, predictions, observations, and discrepancy. Assign separate Smols to model maintenance, sensor ingestion, calibration, anomaly review, and completion. Require human intervention for declared physical safety and equipment-control actions.

21.5 External model role pool

Load several freely available external models through role adapters. Assign the same bounded role to multiple providers using the same kernel pack, canon, examples, and output schema. Compare reliability, cost, latency, intervention rate, and downstream reuse. This demonstrates that role identity can remain independent of any one provider while still exposing implementation differences.

21.6 Bounty demonstration

Publish one narrow bottleneck with a machine-checkable completion contract. Allow humans and agents to submit competing constructions. Settle only after verifier and jury requirements are met. Promote any reusable primitive produced during the attempt, even when the primary bounty remains unresolved.

22. Metrics and Evaluation

The central value proposition becomes testable only when the system measures whether continued use improves future starting conditions. Metrics should remain scoped to a domain, role, federation, or benchmark.

Metric	Interpretation
Construction reuse rate	Fraction of later objects importing previously verified components.
Marginal construction cost	Change in time, compute, human effort, or laboratory expense for repeated task classes.
Verification cost	Resources required to independently check an object or certificate.
Residue visibility	Proportion of known assumptions and discrepancies represented explicitly.
Reproduction rate	Share of objects successfully replayed or reproduced by independent nodes.
Jury defect yield	Actionable defects found per review cost, with false-positive tracking.
Intervention rate	Frequency and type of required human corrections or approvals.
Promotion utility	Measured downstream benefit after a composite becomes a reusable primitive.
Fork survival and return	Continued or renewed use of alternate lineages under changed conditions.
Cross-federation interoperability	Successful exchanges across different registries, canons,

	and role implementations.
Digital-twin residual	Difference between predicted and observed physical behavior under declared calibration.
Bounty resolution quality	Verified completion, cost, time, reuse produced, and unresolved residue.
Provider independence	Ability to replace an external model without losing the role, object schema, or process history.

No metric should be treated as a universal fitness function. A federation should publish the tradeoffs it values and preserve the measurements that support promotion or deprecation decisions.

23. Security, Failure Modes, and Safeguards

A federated constructive ecosystem introduces risks that must be designed into the architecture rather than appended later.

Failure mode	Required response
Unverified fluent output	Quarantine as an uncertified candidate; prevent promotion or settlement.
Canon poisoning or capture	Version canons, preserve maintainers and diffs, support independent forks and multi-jury review.
Sybil or reputation manipulation	Tie standing to reproducible work, task-specific history, costs, and independent verification rather than identity count alone.
Certificate forgery	Use deterministic replay where possible, signatures, verifier diversity, and content-addressed dependencies.
Hidden dependency or data leakage	Require dependency manifests, permissions, privacy labels, and redaction boundaries.
Unsafe physical action	Separate recommendation from control, require hardware interlocks and explicit human authorization.
Runaway self-modification	Restrict changes to sandboxed proposals, trusted-kernel boundaries, staged deployment, and rollback.
Reward gaming	Define completion contracts before work, use independent verifiers, preserve residue, and audit collusion.
Premature canonization	Require reuse evidence, domain-specific metrics, reversible promotion, and retained alternatives.
Central coordinator capture	Make routing decisions inspectable and allow replacement, federation exit, and peer discovery.
External provider drift	Version adapters, benchmark continuously, retain provider provenance, and support failover.
Over-distribution overhead	Measure communication and verification cost; compress repeated structures and keep local work local when appropriate.

Formal limits remain in scope. Distribution does not guarantee resolution of undecidable questions, inaccessible facts, incompatible frames, or tasks whose communication cost dominates their computation. The architecture should record these boundaries rather than reinterpret every failure as insufficient scale.

24. Implementation Roadmap

Implementation should proceed through increasingly complete throughput demonstrations. Each phase should leave behind reusable protocols and objects rather than a one-off prototype.

Phase	Deliverables	Exit condition
0 - Architecture capture	This specification, core schemas, component map, reference terminology.	Independent contributor can identify where a proposed component fits.
1 - Object and role kernel	Primitive registry, role schema, IRRPROOF object, verifier hooks, QED contract.	One local task produces a replayable certified object.
2 - Minimum DST	Advisor, two jurors, canon repository, split verdict support.	Same object receives attributable plural reviews and a scoped completion result.
3 - External model adapters	Provider-agnostic role runtime, bootstrap packs, structured outputs, benchmarks.	At least three distinct services can occupy one role under the same contract.
4 - Federated exchange	Capability manifests, object envelopes, content addressing, peer retrieval, fork lineage.	Two independently operated nodes exchange and verify an object.
5 - Bounty routing	Desired-state contract, reward terms, submission, verifier settlement.	A narrow bounty is resolved or produces reusable partial objects with transparent status.
6 - Topological lab and digital twin	Simulation, apparatus, sensors, calibration, residual tracking, laboratory completion.	A physical run and its twin produce one integrated IRRPROOF package.
7 - Self-tuning roles	Metrics, proposal generation, sandbox evaluation, limited deployment, rollback.	One role improvement demonstrates measured benefit without bypassing review.
8 - Federation growth	Multiple canons, juries, registries, educational pathways, governance dashboards.	Independent communities can fork and interoperate without central permission.

25. Minimum Viable Ecosystem Demonstration

The first complete demonstrator should remain small enough to understand in one sitting while exercising the entire architecture. A recommended scenario uses a bounded mathematical construction or calibrated electrical experiment with a known verification route.

15. A user or bounty issuer declares the objective and completion contract.

16. The seed records environment, permissions, role, and available resources.

17. The task is decomposed into primitive-addressable segments.
18. Three role implementations are instantiated: constructor, verifier, and juror. At least one role uses an external model adapter.
19. The constructor emits an IRRPROOF object with trace, dependencies, and residue.
20. The verifier performs deterministic checks and returns machine-readable results.
21. Two DST jurors operating under distinct canons review the same object.
22. The QED role determines whether the declared completion contract is satisfied.
23. The object, reports, and certificate are exchanged to a second node and replayed.
24. A repeated subconstruction is proposed for promotion; the registry records either adoption, deferral, or rejection with reasons.
25. The dashboard shows cost, interventions, defects, reuse potential, and unresolved work.

The demonstrator succeeds even if the primary scientific claim remains unresolved, provided the system faithfully records where construction stopped, which parts verified, what residue remains, and which reusable objects emerged. The architecture is demonstrated by visible process, not by selecting only tasks guaranteed to end in affirmation.

26. Concluding Specification

The Constructive Ecosystem Architecture defines a federated process through which primitives, roles, models, instruments, canons, juries, experiments, and human purposes can participate in one inspectable throughput. Its distinguishing claim remains narrow and demonstrable: under suitable decomposition, verification, and reuse conditions, continued valid participation can improve the starting vocabulary, protocols, and role implementations available to later participants.

The system does not require a single authoritative intelligence. It distributes constructive work among role-addressable participants, preserves plural review, binds local completion to machine-checkable contracts, and allows federations to fork while retaining interoperability where sufficient interfaces exist. A minimal seed can begin with little more than orientation, grammar, registry, verification, discovery, and human-control capabilities, then bootstrap through compatible systems without representing imported capability as native certainty.

Its value proposition is therefore operational: use the system faithfully, and measure whether the resulting constructions reduce future effort, improve verification, sharpen digital twins, expose residue, create reusable primitives, or support better role protocols. Continued value is earned in use rather than asserted in advance.

Specification statement

A federated constructive ecosystem may increase its future capacity by converting valid, verifiable, and repeatedly useful interactions into reusable structural components while preserving autonomy, provenance, plural review, explicit residue, reversible evolution, and informed human participation.

Appendix A. Core Schemas

A.1 Capability manifest

```
{
  "node_id": "...",
  "kernel_version": "...",
  "registry_namespaces": [...],
  "roles_supported": ["constructor", "verifier"],
  "certificate_formats": [...],
  "resources": {"compute": "...", "tools": [...]},
  "governance": {"human_approval": ["physical_control"]},
  "limitations": [...],
  "terms": {"cost": "...", "license": "..."}
}
```

A.2 Role contract

```
{
  "role_id": "dst-juror",
  "version": "0.1",
  "inputs": ["irrproof_object", "canon"],
  "outputs": ["dst_report"],
  "completion_conditions": ["all required gates addressed"],
  "verification": ["schema", "citation", "consistency"],
  "escalation": ["missing evidence", "unsafe request"],
  "metrics": ["defect_yield", "false_positive_rate"],
  "governance": {"forkable": true}
}
```

A.3 IRRPROOF object skeleton

```
{
  "object_id": "...",
  "question": "...",
  "frame": "...",
  "inputs": [],
  "construction_trace": [],
  "dependencies": [],
  "tests": [],
  "completion_contract": "...",
  "completion_status": "open|partial|complete",
  "residue": [],
  "provenance": {},
  "permissions": {},
  "review_history": []
}
```

A.4 Completion certificate skeleton

```
{
  "object_id": "...",
  "contract_id": "...",
  "checks": [],
  "status": "pass|fail|conditional",
}
```

```

"residue": [],
"verifiers": [],
"jury_reports": [],
"timestamp": "...",
"signatures": [],
"replay_route": "..."
}

```

Appendix B. Protocol Examples

B.1 External model role bootstrap

26. Load the role contract and output schema.
27. Load only the canon sections required for the current task.
28. Provide the local object manifest and unresolved terms.
29. Require the model to declare imported assumptions and abstain where the role contract cannot be satisfied.
30. Run schema and deterministic checks.
31. Return defects to the model for bounded correction.
32. Attach provider, model, adapter, prompt-pack, tool, and version provenance.
33. Submit the candidate output to independent verification before registry admission.

B.2 Promotion protocol

34. Identify a repeated construction or role pattern.
35. Compute structural equivalence across instances.
36. Propose a canonical interface and dependency set.
37. Measure expected reduction in future work.
38. Run compatibility and regression tests.
39. Submit to relevant juries and federation maintainers.
40. Deploy provisionally with rollback support.
41. Observe actual reuse and defects.
42. Promote, revise, fork, or retire according to published metrics.

B.3 Bounty settlement protocol

43. Publish current object, desired state, completion contract, verifier set, reward, deadline, and permissions.
44. Allow participants to register approaches without requiring disclosure beyond the terms.
45. Accept structured submissions containing dependencies, trace, tests, residue, and provenance.
46. Run independent verifiers and required DST juries.
47. Issue a QED certificate for the bounty contract or record conditional/failure status.
48. Settle reward according to the published terms.
49. Preserve reusable partial constructions even when the primary objective remains open.

Appendix C. Open Interfaces and Unresolved Work

This specification intentionally leaves several implementation choices open. They should be resolved through prototypes and measured use rather than fixed prematurely.

- The exact formal language and storage representation for IRRPROOF objects.
- The minimum trusted verification kernel for mixed formal and empirical objects.
- The initial DST canon set and the smallest reproducible traversal example.
- The relationship between content-addressed repositories and any optional distributed ledger.
- The economics and legal form of bounty settlement across jurisdictions.
- The consent, privacy, and intellectual-property defaults for imported external model services.
- The first topological electrical laboratory apparatus and digital-twin benchmark to integrate.
- The minimum local model or deterministic kernel required for the seed to operate without network assistance.
- The compatibility negotiation format among federated primitive namespaces.
- The conditions under which a promoted object can be deprecated without breaking dependent constructions.
- The governance process for high-risk physical, medical, legal, financial, or security-sensitive roles.
- The best metrics for distinguishing useful registry compression from premature abstraction.

These unresolved choices do not prevent implementation of the minimum viable ecosystem. They define the next construction agenda and should become explicit bounties, experiments, role assignments, or protocol forks as the program advances.